

CamoFarmer Documentation

Model Compression Pipeline

Technical Manual

CV10, Center for Computational Imaging and Visual Innovations

De La Salle University

CONTENTS:

Compression Techniques 1

Pruning 1

Knowledge Distillation 9

Backbone Replacement 16

Pruning

prune_helpers

Evaluation and Benchmarking Helpers for Object Detection Models

Provides utility functions for:

- Counting parameters and sparsity
- Benchmarking inference speed
- Computing IoU between bounding boxes
- Evaluating true positive rates on datasets with visualizations

Functions

- **count_parameters(model)**: Count total and non-zero parameters.
 - **benchmark_inference(model, iterations, input_size)**: Measure average inference time.
 - **compute_iou(box1, box2)**: Compute Intersection over Union (IoU) for two boxes.
 - **ccrop_tp(txt_file, model, iou_threshold, viz_folder)**: Evaluate model on file list (CCROP).
 - **dset_tp(image_dir, label_dir, model, iou_threshold, viz_folder)**: Evaluate model on other related datasets.
-

`pruning.prune_helpers.count_parameters(model)`

Count Total and Non-Zero Parameters.

Counts total and non-zero parameters in a PyTorch model for sparsity evaluation.

Parameters

model (`torch.nn.Module`) – The model to evaluate.

Returns

(total_params, non_zero_params)

Return type

tuple

`pruning.prune_helpers.benchmark_inference(model, iterations=100, input_size=(1, 3, 320, 320))`

Benchmark Model Inference Time.

Measures average inference time with GPU synchronization and warmup.

Parameters

- **model** (torch.nn.Module) – The model to benchmark.
- **iterations** (int) – Number of iterations for timing. Default: 100.
- **input_size** (tuple) – Shape of dummy input (B, C, H, W). Default: (1, 3, 320, 320).

Returns

(average_time_ms, std_time_ms)

Return type

tuple

`pruning.prune_helpers.compute_iou(box1, box2)`

Compute Intersection over Union (IoU).

Calculates IoU between two bounding boxes in [x1, y1, x2, y2] format.

Parameters

- **box1** (list or tuple) – [x1, y1, x2, y2] coordinates of first box.
- **box2** (list or tuple) – [x1, y1, x2, y2] coordinates of second box.

Returns

IoU value between 0.0 and 1.0.

Return type

float

`pruning.prune_helpers.ccrop_tp(txt_file, model, iou_threshold, viz_folder='visualizations')`

Evaluate True Positives on CCROP Dataset.

Evaluates model on CCROP dataset using text file of image paths. Creates visualizations.

Parameters

- **txt_file** (str) – Path to .txt file listing image paths.
- **model** – Inference model that supports *model(image_path)* calls.
- **iou_threshold** (float) – IoU threshold for true positive classification.
- **viz_folder** (str) – Directory for visualization images. Default: “visualizations”.

Returns

True positive percentage relative to ground truth labels.

Return type

float

`pruning.prune_helpers.dset_tp(image_dir, label_dir, model, iou_threshold,
viz_folder='visualizations')`

Evaluate True Positives on Generic Dataset.

Evaluates model on standard dataset with separate image/label folders. Creates visualizations.

Parameters

- **image_dir** (str) – Directory containing test images (.jpg, .png).
- **label_dir** (str) – Directory containing corresponding label .txt files.
- **model** – Inference model that supports *model(image_path)* calls.
- **iou_threshold** (float) – IoU threshold for true positive classification.
- **viz_folder** (str) – Directory for visualization images. Default: “visualizations”.

Returns

True positive percentage relative to ground truth labels.

Return type

float

prune_validate

Fine-Tune Pruned Models Script

Loads a previously pruned YOLO or RT-DETR model, validates it, fine-tunes it on the specified dataset, and evaluates its performance after fine-tuning.

Command-line arguments:

- **folder_name** : Name of the folder where pruning results and pruned model are stored.
- **model_type** : Type of model to fine-tune (‘yolo’ or ‘rtdetr’).
- **dset_type** : Dataset type (‘ccrop’, ‘tomatod’, or ‘camocrops’).

Sample usage:

```
>>> python prune_validate.py --folder_name "YOLO_N_CC_HEAD_50" --model_type  
↪ "yolo" --dset_type "ccrop"
```

Outputs:

- Fine-tuned model weights in the results folder.
- Validation results with mAP scores and true positive rates.
- Updated CSV log file with fine-tuning performance metrics.
- Visualization outputs for model predictions.

`pruning.prune_validate.parse_args()`

Parse Command-Line Arguments.

Parses and validates command-line arguments for the fine-tuning script.

Returns

Parsed arguments containing configuration parameters.

Return type

`argparse.Namespace`

`pruning.prune_validate.get_dataset_paths(dset_type)`

Get Dataset Paths Based on Type.

Returns dataset configuration paths for different dataset types.

Parameters

`dset_type` (str) – Dataset type ('ccrop', 'tomatod', or 'camocrops').

Returns

(*data_yaml*, *test_set*, *test_imgs*)

Return type

tuple

`pruning.prune_validate.load_pruned_model(model_type, model_path)`

Load Pruned YOLO or RT-DETR Model.

Loads a previously pruned model for fine-tuning.

Parameters

- **`model_type`** (str) – Either 'yolo' or 'rtdetr'.
- **`model_path`** (str) – Path to the pruned model weights.

Returns

Loaded pruned model instance.

Return type

Model

`pruning.prune_validate.get_training_params(model_type, dset_type)`

Get Training Hyperparameters.

Returns training configuration based on model and dataset type.

Parameters

- **`model_type`** (str) – Either 'yolo' or 'rtdetr'.
- **`dset_type`** (str) – Dataset type ('ccrop', 'tomatod', or 'camocrops').

Returns

Training parameters dictionary.

Return type

dict

`pruning.prune_validate.write_results_csv(results_path, avg_time, std_time, ft_results,
tp_percent)`

Write Fine-Tuning Results to CSV.

Saves fine-tuning performance metrics and statistics to a CSV file.

Parameters

- **results_path** (str) – Path to the CSV file.
- **avg_time** (float) – Average inference time.
- **std_time** (float) – Standard deviation of inference time.
- **ft_results** – Fine-tuning validation results.
- **tp_percent** (float) – True positive percentage.

`pruning.prune_validate.main()`

Main Entry Point for Fine-Tuning Script.

ultralytics_pruner

Pruning Script for YOLO and RT-DETR Models

Performs L1 unstructured pruning on YOLO or RT-DETR models. It allows pruning specific groups of layers (backbone, neck, head, or all convolutional layers) and saves the pruned model along with pruning statistics.

Command-line arguments:

- **folder_name** : Folder name for storing pruning results.
- **bmodel_path** : Path to the baseline model weights.
- **model_type** : Type of model to prune ('yolo' or 'rt detr').
- **prune_group** : Layer group to prune ('backbone', 'neck', 'head', 'all').
- **prune_ratio** : L1 unstructured pruning ratio (float).

Sample usage:

```
>>> python ultralytics_pruner.py --folder_name "YOLO_N_CC_HEAD_50" --bmodel_
↳ path "runs/detect/ccrop_300n_coare/weights/best.pt" --model_type "yolo" --
↳ prune_group "head" --prune_ratio .50
```

Outputs:

- Pruned model (.pt file) in the results folder.
- CSV log file containing pruning statistics.

`pruning.ultralytics_pruner.parse_args()`

Parse Command-Line Arguments.

Parses and validates command-line arguments for the pruning script.

Returns

Parsed arguments containing all configuration parameters.

Return type

`argparse.Namespace`

`pruning.ultralytics_pruner.load_model(model_type, model_path)`

Load YOLO or RT-DETR Model.

Loads the specified model type and configures layer group slices for pruning.

Parameters

- **model_type** (str) – Either ‘yolo’ or ‘rt detr’.
- **model_path** (str) – Path to the model weights.

Returns

(base_model, inner_model, group_slices)

Return type

tuple

`pruning.ultralytics_pruner.prune_model(model, prune_group, prune_ratio, group_slices)`

Apply L1 Unstructured Pruning.

Applies L1 unstructured pruning to specified layer groups in the model.

Parameters

- **model** (torch.nn.Module) – The model to prune.
- **prune_group** (str) – Layer group (‘backbone’, ‘neck’, ‘head’, or ‘all’).
- **prune_ratio** (float) – Proportion of weights to prune.
- **group_slices** (dict) – Slice indices for each group.

Returns

Pruned model.

Return type

`torch.nn.Module`

`pruning.ultralytics_pruner.write_results_csv(results_path, data)`

Write Pruning Results to CSV.

Saves pruning statistics and performance metrics to a CSV file.

Parameters

- **results_path** (str) – Path to the CSV file.
- **data** (list) – List of data rows to write.

`pruning.ultralytics_pruner.main()`

Main entry point for pruning script.

main_prune

Main Pruning Pipeline for SSD.

Provides the main training and testing pipeline for pruned SSD models with support for multiple datasets and architectures.

Functions:

- **zip_directory**: Store project files in a zip archive.
- **save_config**: Save experiment configuration.
- **string_to_boolean**: Convert string to boolean value.
- **main**: Main execution function for training/testing.

`pruning.main_prune.zip_directory(path, zip_file)`

Store all py and cfg project files inside a zip file.

Parameters

- **path** (str) – Current path to scan for files.
- **zip_file** (zipfile.ZipFile) – Zip file to contain the project files.

`pruning.main_prune.save_config(path, version, config)`

Save the configuration of the experiment.

Parameters

- **path** (str) – Save path for configuration files.
- **version** (str) – Version of the model based on timestamp.
- **config** (dict) – Contains argument and its value.

Returns

Version string used for file naming.

Return type

str

`pruning.main_prune.string_to_boolean(v)`

Convert string to boolean.

Parameters

v (str) – String representation of a boolean value; must be 'true' or 'false'.

Returns

Boolean true or false.

Return type

bool

`pruning.main_prune.main(version, config, output_txt)`

Main execution function for training/testing.

Parameters

- **version** (str) – Version identifier for the experiment.
- **config** – Configuration object containing all parameters.
- **output_txt** (str) – Path to output text file for logging.

solver_prune

Pruning Script for SSD

This module provides the main training and evaluation solver for SSD models with integrated L1 unstructured pruning capabilities.

Classes:

Solver: Main solver class for training and evaluating pruned SSD models.

class pruning.solver_prune.Solver

Bases: object

Main solver for training and evaluating SSD models with L1 unstructured pruning.

Provides comprehensive functionality for training, evaluation, and pruning of SSD models on various datasets.

DEFAULTS = {}

count_nonzero_params(*model*)

Count the number of non-zero parameters in the model.

Parameters

model – PyTorch model to count parameters for.

Returns

Number of non-zero parameters with `requires_grad=True`.

Return type

int

save_pruned_model(*model*, *path*)

Save pruned model to the specified path.

Parameters

- **model** – PyTorch model to save.
- **path** (str) – File path to save the model state dictionary.

apply_pruning(*model*, *prune_amount=0.1*, *mode=None*)

Apply L1 unstructured pruning to Conv2d layers of SSD300.

Parameters

- **model** – The SSD model to prune.
- **prune_amount** (float, optional) – Proportion of weights to prune. Defaults to 0.1 (10%).

- **mode** (str, optional) – Target component for pruning. Options are ‘backbone’, ‘neck’, ‘head’, or None/’all’ for entire model. Defaults to None.

Raises

ValueError – If mode is not one of the supported options.

build_model()

Instantiate the model, loss criterion, and optimizer.

Initializes anchor boxes, model architecture, loss function, and optimizer based on the configuration settings.

print_params(model)

Prints the structure of the network and the total number of parameters + GFLOPs

print_network(model)

Prints the structure of the network and the total number of parameters + GFLOPs.

adjust_learning_rate(optimizer, gamma, step)

Sets the learning rate to the initial LR decayed by 10 at every specified step.

print_loss_log(start_time, iters_per_epoch, e, i, class_loss, loc_loss, loss)

Prints the loss and elapsed time for each epoch.

save_model(e)

Saves a model per epoch.

model_step(images, targets, count)

Signifies a step for each iteration.

train()

Training process for SSD.

eval(dataset, max_per_image, score_threshold)**test()**

Testing process for SSD.

Knowledge Distillation

RT-DETR Knowledge Distillation

distill_model

RT-DETR Knowledge Distillation Model.

Custom RT-DETR class with knowledge distillation for lightweight object detection.

Classes:

RTDETRDistillation: Extended RT-DETR class with distillation trainer.

class knowledge_distillation.rtdetr.distill_model.RTDETRDistillation

Bases: RTDETR

RT-DETR with Knowledge Distillation.

Extended RT-DETR class supporting teacher-student distillation training.

Example

```
>>> student = RTDETRDistillation("efficientnet_b0.yaml")
>>> student.train(data="dataset.yaml", teacher_weights="teacher.pt", epochs=100)
```

property task_map

Define Task Mapping for RT-DETR Distillation.

Returns

Task components with custom distillation trainer.

Return type

dict

distill_train

class knowledge_distillation.rtdetr.distill_train.RTDETRDistillationTrainer

Bases: RTDETRTrainer

Extended RT-DETR Trainer with Knowledge Distillation.

Implements teacher-student training with feature alignment for RT-DETR models.

criterion(preds, batch)

Compute Combined Detection and Distillation Loss.

Parameters

- **preds** – Student model predictions.
- **batch** – Training batch data.

Returns

(combined_loss, loss_items) with detection + distillation losses.

Return type

tuple

validate()

Override to disable teacher during validation to reduce FLOPS.

Returns

Dictionary of validation metrics. fitness (float): Fitness score for the validation.

Return type

metrics (*dict*)

```
label_loss_items(loss_items=None, prefix='train')
```

YOLOv8 Knowledge Distillation

distill_model

YOLOv8 Knowledge Distillation Model.

Custom YOLOv8 class with feature-based knowledge distillation for model compression.

Classes:

YOLOv8Distillation: Extended YOLO class with distillation trainer.

```
class knowledge_distillation.yolov8.distill_model.YOLOv8Distillation
```

Bases: YOLO

YOLOv8 with Knowledge Distillation.

Extended YOLO class supporting teacher-student distillation training.

Example

```
>>> model = YOLOv8Distillation('yolov8n.yaml')
>>> model.train(data='coco.yaml', teacher_model='yolov8l.pt', epochs=100)
```

property task_map

Define Task Mapping for Distillation.

Returns

Task components with custom distillation trainer.

Return type

dict

distill_train

```
class knowledge_distillation.yolov8.distill_train.YOLOv8DistillationTrainer
```

Bases: DetectionTrainer

Extended Detection Trainer with Knowledge Distillation.

Implements teacher-student training with feature alignment and logit distillation.

```
label_loss_items(loss_items=None, prefix='train')
```

Return a loss dict with labeled training loss items tensor.

Parameters

- **loss_items** (List[float], optional) – List of loss values.
- **prefix** (str) – Prefix for keys in the returned dictionary.

Returns

Dictionary of labeled loss items if `loss_items` is provided, otherwise list of keys.

Return type

(*Dict* | *List*)

read_results_csv()

Read results.csv into a dictionary using pandas.

SSD Knowledge Distillation

main_kd

SSD Knowledge Distillation Main Training Script.

Entry point for training SSD models with knowledge distillation. Contains comprehensive argument parsing, configuration management, and training orchestration for teacher-student training paradigms on multiple datasets.

Functions:

- **zip_directory**: Archive project files for experiment tracking.
- **save_config**: Save experimental configuration and create project archive.
- **string_to_boolean**: Convert string representations to boolean values.
- **main**: Main training function orchestrating the knowledge distillation process.

`knowledge_distillation.ssd.main_kd.zip_directory(path, zip_file)`

Store all Python and configuration files in a zip archive.

Archives all .py and .cfg files from the specified path for experiment tracking and reproducibility purposes. Configuration files are removed after archiving.

Parameters

- **path** (str) – Directory path to search for project files.
- **zip_file** (zipfile.ZipFile) – Zip file object to store the project files.

`knowledge_distillation.ssd.main_kd.save_config(path, version, config)`

Save experimental configuration and create project archive.

Saves the configuration parameters to a .cfg file and creates a zip archive containing all project files for experiment reproducibility and tracking.

Parameters

- **path** (str) – Save path for configuration and archive files.
- **version** (str) – Version identifier based on timestamp for file naming.
- **config** (dict) – Dictionary containing all configuration parameters.

Returns

The version string used for file naming.

Return type

str

knowledge_distillation.ssd.main_kd.**string_to_boolean**(v)

Convert string to boolean value.

Converts string inputs to boolean values for command-line argument parsing.

Parameters

v (str) – String representation of boolean value.

Returns

Boolean value corresponding to the string input.

Return type

bool

knowledge_distillation.ssd.main_kd.**main**(version, config, output_txt)

Main training function for SSD knowledge distillation.

Starts the knowledge distillation training process including data loading, solver initialization, and training/testing execution based on the specified mode. Enables fast training with cudnn.benchmark optimization.

Parameters

- **version** (str) – Version identifier for the training run.
- **config** – Configuration object containing all training parameters.
- **output_txt** (str) – Path to output text file for logging.

solver_kd

SSD Knowledge Distillation Solver Module.

Provides the solver for SSD knowledge distillation training, implementing teacher-student training with response-based knowledge transfer and feature mimicking capabilities.

Classes:

Solver: Main solver class for SSD knowledge distillation training and evaluation.

class knowledge_distillation.ssd.solver_kd.**Solver**

Bases: object

Main solver for SSD knowledge distillation training and evaluation.

Implements teacher-student training paradigms with response-based knowledge transfer and feature mimicking capabilities for SSD models. Supports multiple datasets and flexible knowledge distillation configurations.

Features:

- Teacher and student model initialization

- Knowledge distillation training with configurable temperature
- Multi-dataset evaluation (VOC, COCO, TomatoD, CCRP, CamoCrops)
- Performance benchmarking and detailed metrics
- Per-class evaluation for agricultural datasets

DEFAULTS = {}

build_model()

Instantiate student and teacher models, loss criterion, and optimizer.

Initializes anchor boxes, student model architecture, loss function, optimizer, learning rate scheduler, and teacher model for knowledge distillation if enabled.

print_network(model)

Print network architecture and computational statistics.

Parameters

model – PyTorch model to analyze and print statistics for.

adjust_learning_rate(optimizer, gamma, step)

Sets the learning rate to the initial LR decayed by 10 at every specified step.

Parameters

- **optimizer** – PyTorch optimizer to adjust learning rate for.
- **gamma** (float) – Decay factor for learning rate reduction.
- **step** (int) – Current step/epoch for learning rate calculation.

print_loss_log(start_time, iters_per_epoch, e, i, class_loss, loc_loss, loss)

Print formatted training loss log with timing information.

Displays current epoch progress, loss values, and estimated remaining time for training completion.

Parameters

- **start_time** (float) – Training start timestamp.
- **iters_per_epoch** (int) – Number of iterations per epoch.
- **e** (int) – Current epoch number.
- **i** (int) – Current iteration within epoch.
- **class_loss** – Classification loss value.
- **loc_loss** – Localization loss value.
- **loss** – Total combined loss value.

save_model(*e*)

Save model checkpoint after training epoch.

Saves a model per epoch to enable resuming training and model evaluation.

Parameters

e (int) – Current epoch number (0-indexed).

model_step(*images*, *targets*, *count*)

Perform one training iteration with knowledge distillation.

Parameters

- **images** – Batch of input images for training.
- **targets** – Ground truth targets for the batch (bounding boxes and classes).
- **count** – Batch counter (unused in current implementation).

Returns

(class_loss, loc_loss, total_loss, count) for logging purposes.

Return type

tuple

train()

Execute the knowledge distillation training process.

Trains the student model using knowledge distillation from the teacher model (if enabled), with proper learning rate scheduling, model checkpointing, and comprehensive logging of training progress.

eval(*dataset*, *max_per_image*, *score_threshold*)

Evaluate the trained model on the specified dataset.

Performs comprehensive evaluation including detection, NMS, and dataset-specific metrics calculation.

Parameters

- **dataset** – Dataset object for evaluation.
- **max_per_image** (int) – Maximum number of detections per image.
- **score_threshold** (float) – Minimum confidence score for detections.

test()

Execute model testing and evaluation.

Sets the model to evaluation mode and performs comprehensive testing on the loaded dataset using the configured evaluation parameters.

Backbone Replacement

For backbone replacement, the yaml files are found in the *backbone_replacement* directory.